

Grupo: CITIM 21 - 14

Integrantes:

Christian Sanchez Alvarez
Mikkel Alexander Ushiña Andrango
Mario Lorenzo de la Losa
Yzan Martinez Manzano
Alejandra María Muñoz Fandiño

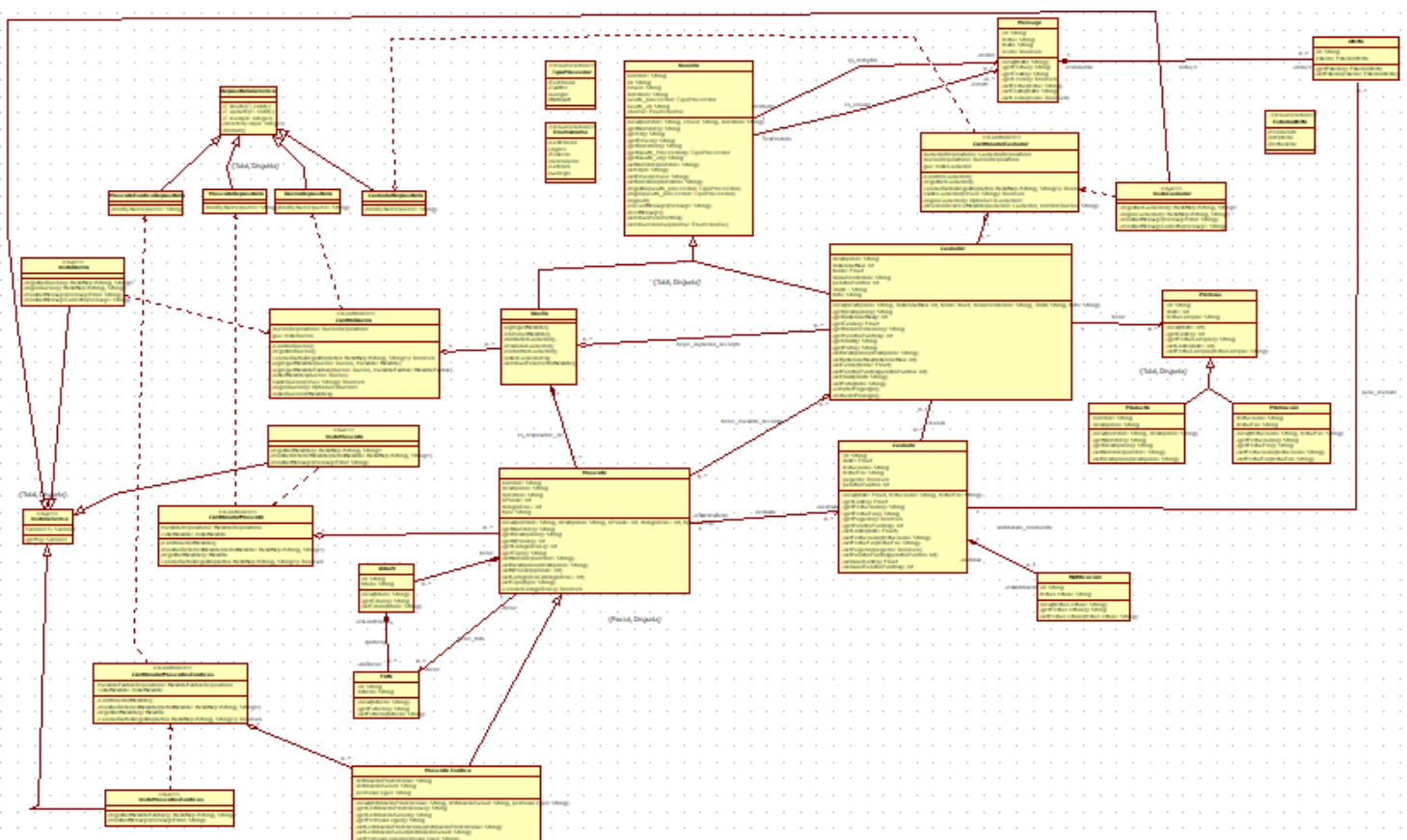
Redmine: <https://fis.etsisi.upm.es/projects/citim21-14-paseando-a-pancho>

Gitlab: <https://gitlab.etsisi.upm.es/br0285/citim21-14-paseando-a-pancho>

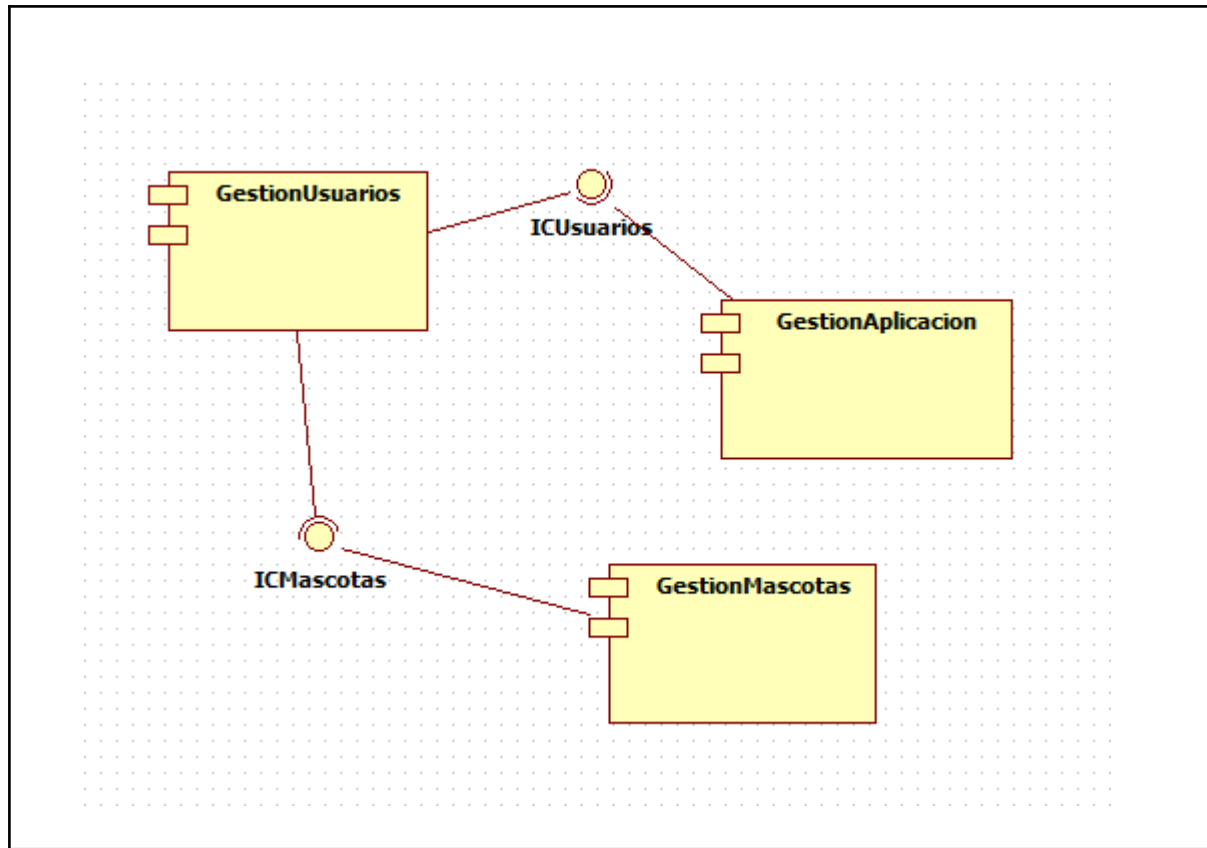
1. DISEÑO

Líderes de esta fase : Christian, Mario y Alejandra

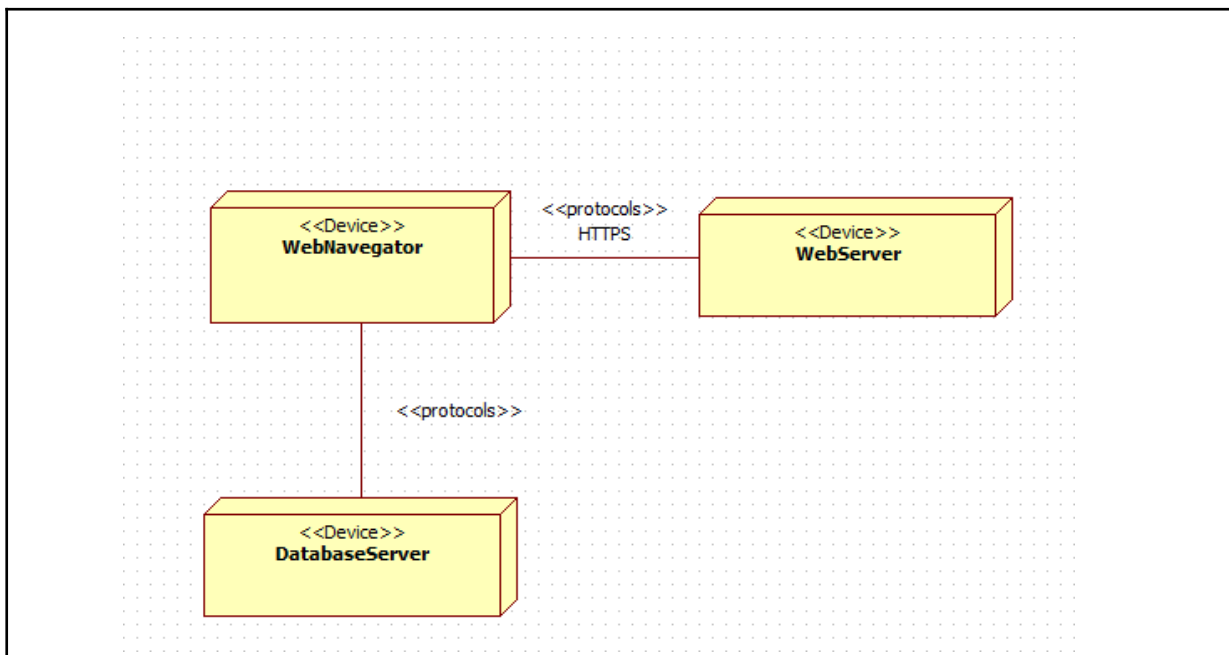
A. Diagrama de clases de diseño



B. Diagrama de componentes



C. Diagrama de despliegue



2. IMPLEMENTACIÓN

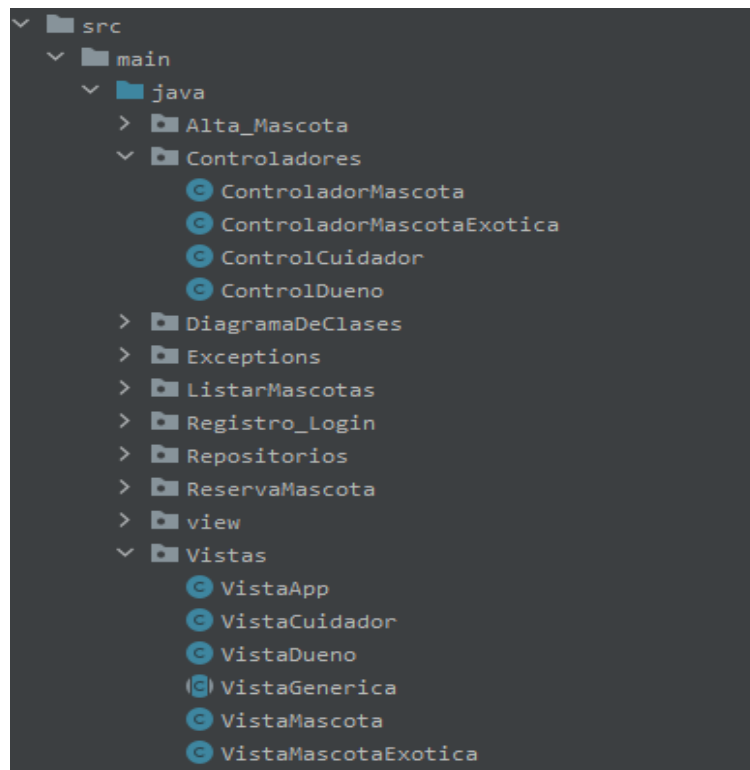
Líderes de esta fase: Yzan y Mikkel

A. Generación automática de código

El código se ha creado automáticamente a partir del diagrama de clases de diseño.

```
//  
//  
//  Generated by StarUML(tm) Java Add-In  
//  
//  @ Project : CuidandoAPancho  
//  @ File Name : Usuario.java  
//  @ Date : 03/05/2024  
//  @ Author :  
//  
//
```

B. Código fuente



C. Valoración crítica

Tras realizar la fase de especificación de requisitos pudimos conocer más sobre sus funcionalidades de la aplicación de “Cuidando a Pancho” mediante

la documentación y los diagramas de clases. En esta fase, se estudiaron las relaciones entre los elementos, el comportamiento y los casos de usos, lo cual sirvió de base para el diagrama de clases de diseño, componentes e implementación. Esto nos permitió tener una vista general a lo más específico. También identificar los fallos y mejorar el diseño.

3. VERIFICACIÓN Y VALIDACIÓN

Líderes de esta fase: Christian y Alejandra

A. Pruebas unitarias

Vamos a realizar las pruebas unitarias de caja negra con las funcionalidades que nos dió el profesor en el principio de la segunda entrega que son: Registro/Login, AltaMascota/exotica, ListarMascotas y CreacionReservaMascota(Dueño).

En este caso hemos elegido la funcionalidad Registro/Login.

Pruebas de caja negra sobre la clase dueño

TABLA CASOS EQUIVALENCIA Y CASOS POSIBLES

Clase Dueño (Dueno)

ENTRADA	CLASES DE EQUIVALENCIA VÁLIDA	CLASES DE EQUIVALENCIA NO VÁLIDA
NOMBRE	V1.Nombre no vacío	N1.Nombre vacío
EMAIL	V2.Email con formato correcto V3.Email no existente en la base de datos V4.Email no vacío	N2.Email con formato incorrecto N3.Email existente en la base de datos N4.Email vacío
CONTRASEÑA	V5.Contraseña no vacía	N5.Contraseña vacía
DIRECCIÓN	V6.Dirección no vacía	N6.Dirección vacía

CP1: V1, V2, V5, V6

CP2: V1, V3, V5, V6

CP3: V1, V4, V5, V6

```
Por favor, introduce los datos para el registro del Dueno:  
Nombre: Cristian  
Email: cristian@gmail.com  
Contraseña: jajaja  
Dirección: calle 13
```

CP4: V1, V2, V5, N6

```
Por favor, introduce los datos para el registro del Dueno:  
Nombre: cristian  
Email: hola@gmail.com  
Contraseña: jajaja  
Dirección:  
No se ha podido realizar el login, intentelo de nuevo
```

CP5: V1, V2, N5, V6

```
Por favor, introduce los datos para el registro del Dueno:  
Nombre: cristian  
Email: hola@gmail.com  
Contraseña:  
Dirección: calle 14  
No se ha podido realizar el login, intentelo de nuevo
```

CP6: V1, N2, V5, V6

```
Por favor, introduce los datos para el registro del Dueno:  
Nombre: cristian  
Email: adsdasdad  
Contraseña: asdjadja  
Dirección: calle 13  
No se ha podido realizar el login, intentelo de nuevo
```

CP7: V1, N3, V5, V6

```
Por favor, introduce los datos para el registro del Dueno:  
Nombre: cristian  
Email: cristian@gmail.com  
Contraseña: lololo  
Dirección: calle 13  
No se ha podido realizar el login, intentelo de nuevo
```

CP8: V1, N4, V5, V6

```
Por favor, introduce los datos para el registro del Dueno:
Nombre: cristian
Email:
Contrasenia: asfas
Dirección: calle 13
No se ha podido realizar el login, intentelo de nuevo
```

CP9: N1, V2, V5, V6

```
Por favor, introduce los datos para el registro del Dueno:
Nombre:
Email: adade@gmail.com
Contrasenia: asdadw
Dirección: doctor criado
No se ha podido realizar el login, intentelo de nuevo
```

Para ver que se comprueba de forma correcta con todos sus métodos en Registro/Login:

```
Elije como desea interactuar con la app:
exit
Dueno
Cuidador

Dueno
Elije la opcion que desees:
exit
Registrarme
Logearme

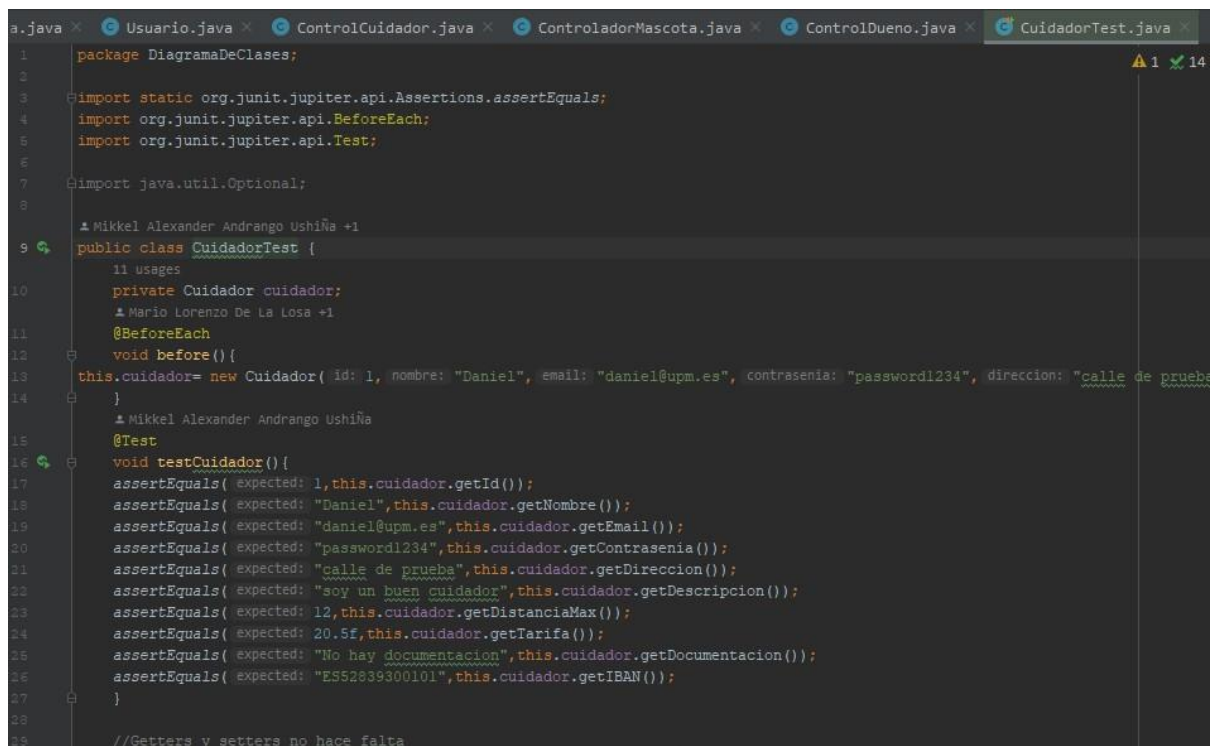
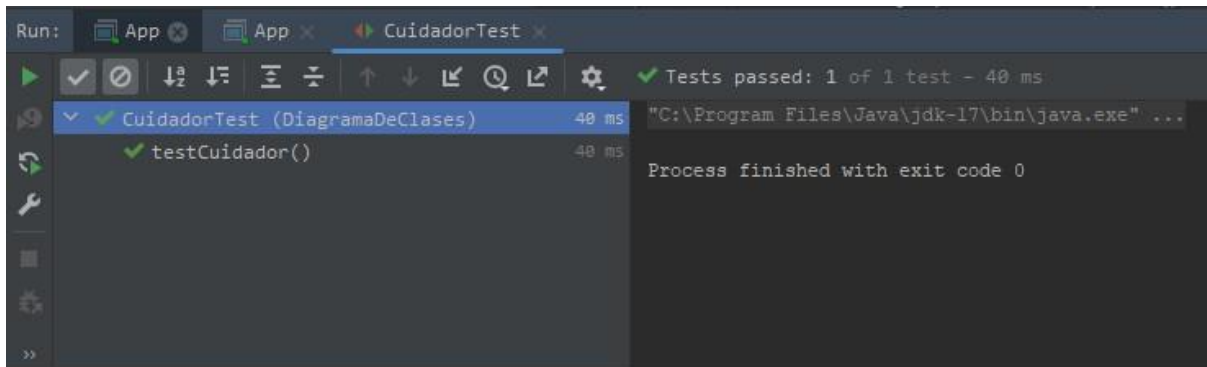
Registrarme
Por favor, introduce los datos para el registro del Dueno:
Nombre: Alejandra
Email: alba@alumnos.upm.es
Contrasenia: 1234
Dirección: Atocha 12
Elije la opcion que desees:
exit
Registrarme
Logearme

Logearme
Por favor, introduce tus credenciales para iniciar sesión:
Email: alba@alumnos.upm.es
Contrasenia: 1234
Elije la opcion que desees:
exit
Logout
ListarMascotas
AltaMascota
AltaMascotaExotica
```

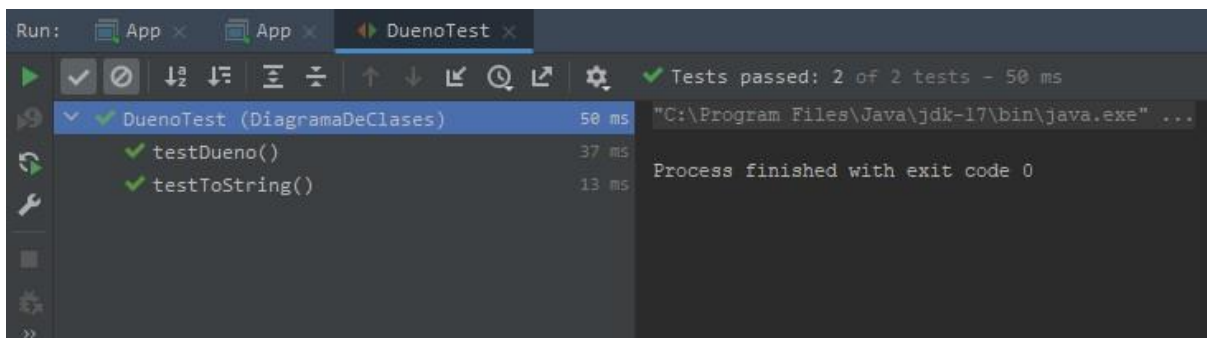
B. Pruebas de aceptación

Aquí adjuntamos todos los tests y los códigos de la funcionalidad Registro/Login con captura de pantalla:

CuidadorTest:



DuenoTest:



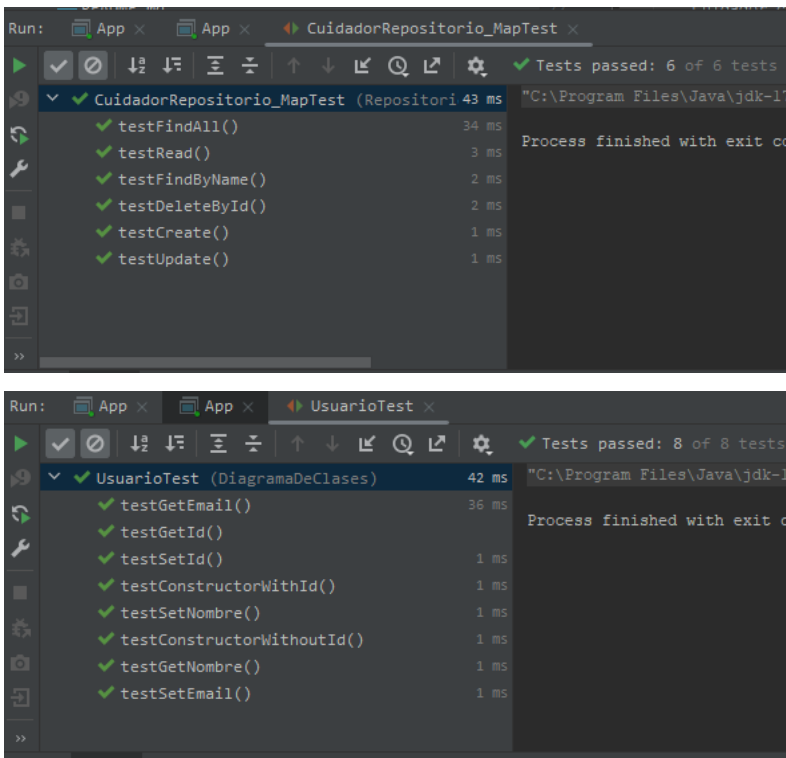
```
controlCuidador.java x ControladorMascota.java x ControlDueno.java x CuidadorTest.java x DuenoTest.java x
1 package DiagramaDeClases;
2
3 import ...
4
5
6
7
8 class DuenoTest {
9     9 usages
10     private Dueno dueno;
11     Mikkell Alexander Andrango Ushiña
12     @BeforeEach
13     void before() { this.dueno = new Dueno( id: 1, nombre: "Juan", email: "juan@upm.es", contrasenia: "123", direccion: "cas
14     Mikkell Alexander Andrango Ushiña
15     @Test
16     void testDueno() {
17         assertEquals( expected: 1, this.dueno.getId());
18         assertEquals( expected: "Juan", this.dueno.getNombre());
19         assertEquals( expected: "juan@upm.es", this.dueno.getEmail());
20         assertEquals( expected: "123", this.dueno.getContrasenia());
21         assertEquals( expected: "casal", this.dueno.getDireccion());
22         this.dueno.setNombre("Juan Carlos");
23         assertEquals( expected: "Juan Carlos", this.dueno.getNombre());
24     }
25 }
```

ControlDueno Test:

```
Run: App x App x ControlDuenoTest.agregarMascota_conMascotaValida... x
Tests passed: 1 of 1 test - 70 ms
ControlDuenoTest (Controladores) 70 ms
agregarMascota_conMascotaValida_agregaMascotaAlDueno 70 ms
Process finished with exit code 0
```

```
ControlDuenoTest.agregarMascota_conMascotaValida_agregaMascotaAlDueno
CuidadorRepositorio_MapTest.java x UsuarioTest.java x ControlCuidadorTest.java x ControlDuenoTest.java x
1 package Controladores;
2
3 import ...
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20 public class ControlDuenoTest {
21     2 usages
22     private ControlDueno controlDueno;
23
24
25     Mikkell Alexander Andrango Ushiña +1
26     @BeforeEach
27     public void setUp() { controlDueno = new ControlDueno(); }
28
29
30
31     Yzan M M
32     @Test
33     public void agregarMascota_conMascotaValida_agregaMascotaAlDueno() {
34         // Crear un dueño y una mascota válidos
35         Dueno dueno = new Dueno( nombre: "Juan Pérez", email: "juan.perez@email.com", contrasenia: "password123", d
36         Mascota mascota = new Mascota( codigORIAC: 1L, nombre: "Max", descripcion: "Perro de raza Golden Retriever",
```


También hemos comprobado todos los tests y funcionan correctamente, en otras clases (por si acaso tenemos que documentar también) adjuntamos captura de pantalla:



C. Trazabilidad

Fase de definición de requisitos

1. Documentación en Redmine sobre el requisito funcional registro/login()

Requisito_Funcional #18070

ModificarTiempo dedicadoMonitorizarCopiarBorrar

Registro con Google

« Anterior | 8/34 | Siguiente »

Añadido por Cristian Álvarez Sánchez hace 3 meses. Actualizado hace 3 meses.

Estado: Especificando

Prioridad: Alta

Asignado a: -

Fecha de inicio: 2024-03-09

Fecha fin:

Subtareas

Añadir

Peticiones relacionadas

Añadir

Registro con Google (#18070)

- Introducción: los usuarios deben registrar con su correo de Google en la aplicación "Cuidando a Pancho".
- Entradas: se selecciona las dos opciones en forma de casilla: responsable o cuidador.
- Proceso: se recogerá la opción elegida.
- Salidas: se guardará la opción elegida, dando de alta como "dueño de mascota" o "cuidador" según la opción que eligió durante el registro.

Login con Google

« Anterior | 30/34 | Siguiente »

Añadido por Alejandra María Muñoz Fandiño hace 3 meses. Actualizado hace 3 meses.

Estado:

Especificando

Fecha de inicio:

2024-03-02

Prioridad:

Alta

Fecha fin:

Asignado a:

-

Subtareas

Añadir

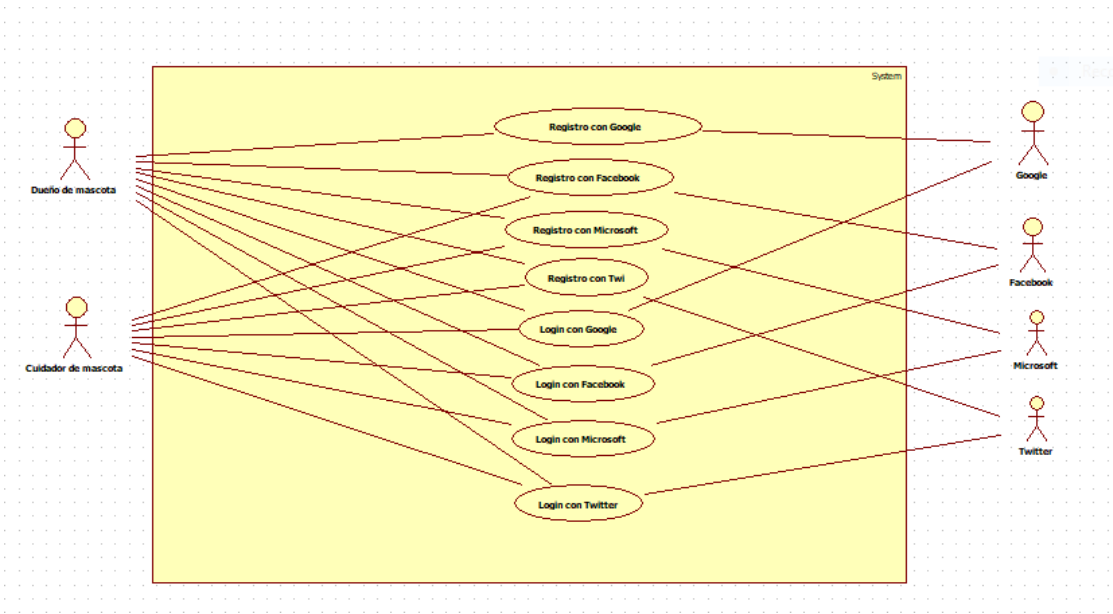
Peticiones relacionadas

Añadir

Login con Google (#17549)

- Introducción: Los usuarios (dueños de mascotas y cuidadores) introducirán su correo de Google para poder registrarse, iniciar y dar de alta.
- Entradas: durante el proceso de login, tendrán dos opciones: quiero que cuiden a mi mascota o quiero cuidar mascotas.
- Proceso: se recogerá y se guardará la opción elegida.
- Salidas: la opción elegida se almacenará en la base de datos.

2. Caso de uso generado para la gestión de usuarios donde está incluida la funcionalidad registro/login



3. Caso de uso extendido para el registro/login

En este caso, se puede poner cualquier servicio o plataforma de correo electrónico: Twitter, Facebook, Google y Microsoft

Nombre:	Registro con Twitter (#17291)	
Actores:	Usuario, Twitter	
Objetivo:	El usuario puede registrarse mediante esta red social y acceder a las funcionalidades de la aplicación.	
Descripción:	El usuario proporciona su cuenta de Twitter y se registrará correctamente.	
Tipo:	Primario	
Precondiciones:	Ninguna	
Dependencias:	Ninguna	
Secuencia Normal:	Nº Paso	Acción
	1	El usuario elige registrarse con Twitter
	2	El sistema redirige al usuario a Twitter
	3	Twitter le pide al usuario que introduzca usuario y contraseña
	4	El usuario introduce su usuario y contraseña

	5	Twitter comprueba que el usuario y contraseña son correctos
	6	Twitter notifica que los datos son correctos
	7	El usuario se ha registrado correctamente y puede acceder a las funcionalidades del programa2
Secuencias Alternativas:	Nº Paso	Acción
	6.1	Twitter notifica que los datos no son correctos
	6.2	Se vuelve al paso 3 de la secuencia normal
Postcondiciones:	El usuario se ha registrado correctamente y ahora puede usar todas las funcionalidades de la aplicación	
Importancia:	Elevada	
Comentarios:	Esto funciona igual para todos los tipos de registro, con la diferencia de que el sistema redirige al usuario a las diferentes páginas web para poder registrarse, es decir a Twitter, Facebook, Google o Microsoft	

Nombre:	Login con Twitter (#17547)	
Actores:	Usuario, Twitter	
Objetivo:	El usuario puede usar las funcionalidades de la aplicación.	
Descripción:	El usuario proporciona su cuenta de Twitter y podrá utilizar el sistema.	
Tipo:	Primario	
Precondiciones:	El usuario debe haberse registrado en la aplicación	
Dependencias:	Ninguna	
Secuencia Normal:	Nº Paso	Acción
	1	El usuario elige hacer el login con Twitter
	2	El sistema redirige al usuario a Twitter
	3	Twitter le pide al usuario que introduzca usuario y contraseña
	4	El usuario introduce su usuario y contraseña
	5	Twitter comprueba que el usuario y contraseña son correctos
	6	Twitter notifica que los datos son correctos
	7	El usuario se ha registrado correctamente y puede acceder a las funcionalidades del programa2
Secuencias Alternativas:	Nº Paso	Acción
	6.1	Twitter notifica que los datos no son correctos
	6.2	Se vuelve al paso 3 de la secuencia normal
Postcondiciones:	El usuario puede usar todas las funcionalidades de la aplicación	
Importancia:	Elevada	
Comentarios:	Esto funciona igual para todos los tipos de login, con la diferencia de que el sistema redirige al usuario a las diferentes páginas web para poder registrarse, es decir Twitter, Facebook, Google y Microsoft	

Fase de implementación

1. Código fuente de registro/login

Clase

```
package Registro_Login;

no usages  ▲ Yzan M M
public class RegistroYLoginDeUsuario {

    // Atributos para el registro
    6 usages
    private String nombreUsuario;
    4 usages
    private String contrasena;
    3 usages
    private String correoElectronico;

    // Atributos para el login
    3 usages
    private String nombreUsuarioLogin;
    3 usages
    private String contrasenaLogin;

    no usages  ▲ Yzan M M
    public RegistroYLoginDeUsuario(String nombreUsuario, String contrasena, String correoElectronico) {
        this.nombreUsuario = nombreUsuario;
        this.contrasena = contrasena;
        this.correoElectronico = correoElectronico;
    }
}
```

```
no usages  ▲ Yzan M M
public String getNombreUsuario() {
    return nombreUsuario;
}

no usages  ▲ Yzan M M
public void setNombreUsuario(String nombreUsuario) {
    this.nombreUsuario = nombreUsuario;
}

no usages  ▲ Yzan M M
public String getContrasena() {
    return contrasena;
}

no usages  ▲ Yzan M M
public void setContrasena(String contrasena) {
    this.contrasena = contrasena;
}

no usages  ▲ Yzan M M
public String getCorreoElectronico() {
    return correoElectronico;
}

no usages  ▲ Yzan M M
public void setCorreoElectronico(String correoElectronico) {
    this.correoElectronico = correoElectronico;
}
}
```

```

no usages  ▲ Yzan M M
public String getNombreUsuarioLogin() {
    return nombreUsuarioLogin;
}

no usages  ▲ Yzan M M
public void setNombreUsuarioLogin(String nombreUsuarioLogin) {
    this.nombreUsuarioLogin = nombreUsuarioLogin;
}

no usages  ▲ Yzan M M
public String getContraseñaLogin() {
    return contraseñaLogin;
}

no usages  ▲ Yzan M M
public void setContraseñaLogin(String contraseñaLogin) {
    this.contraseñaLogin = contraseñaLogin;
}

// Registrar nuevo usuario
no usages  ▲ Yzan M M
public void registrarUsuario() {
    System.out.println("Usuario registrado: " + nombreUsuario);
}

```

```

// Login a un usuario
no usages  ▲ Yzan M M
public boolean iniciarSesion() {
    if (nombreUsuarioLogin.equals(nombreUsuario) && contraseñaLogin.equals(contraseña)) {
        System.out.println("Inicio de sesión exitoso para: " + nombreUsuario);
        return true;
    } else {
        System.out.println("Credenciales incorrectas. Intente de nuevo.");
        return false;
    }
}
}

```

2. Vista de registro/login

Clase VistaDueno:

```
package Vistas;

import java.util.HashMap;

4 usages: 1 Mikkel Alexander Andrango Ushiña
public class VistaDueno extends VistaGenerica{
    1 usage: 1 Mikkel Alexander Andrango Ushiña
    public HashMap<String, String> registrarDueno() {
        HashMap<String, String> datosRegistro = new HashMap<>();
        System.out.println("Por favor, introduce los datos para el registro del Dueno:");
        System.out.print("Nombre: ");
        datosRegistro.put("Nombre", getS().nextLine());
        System.out.print("Email: ");
        datosRegistro.put("Email", getS().nextLine());
        System.out.print("Contraseña: ");
        datosRegistro.put("Contraseña", getS().nextLine());
        System.out.print("Dirección: ");
        datosRegistro.put("Direccion", getS().nextLine());
        return datosRegistro;
    }

    1 usage: 1 Mikkel Alexander Andrango Ushiña
    public HashMap<String, String> loginDueno() {
        HashMap<String, String> datosLogin = new HashMap<>();
        System.out.println("Por favor, introduce tus credenciales para iniciar sesión:");
        System.out.print("Email: ");
        datosLogin.put("Email", getS().nextLine());
        System.out.print("Contraseña: ");
        datosLogin.put("Contraseña", getS().nextLine());
        return datosLogin;
    }
}
```

```
2 usages: 1 Mikkel Alexander Andrango Ushiña
public void mostrarMensaje(String mensajeError) {
    System.out.println(mensajeError);
}

5 usages: 1 Mikkel Alexander Andrango Ushiña
public void mostrarMensajeCorrecto(String mensaje) {
    System.out.println(mensaje);
}
}
```


Clase VistaCuidador:

```
package Vistas;

import java.util.HashMap;

4 usages  ▲ Mikkell Alexander Andrango Ushiña
public class VistaCuidador extends VistaGenerica{
    1 usage  ▲ Mikkell Alexander Andrango Ushiña
    public HashMap<String, String> registrarCuidador() {
        HashMap<String, String> datosRegistro = new HashMap<>();
        System.out.println("Por favor, introduce los datos para el registro del cuidador:");
        System.out.print("Nombre: ");
        datosRegistro.put("Nombre", getS().nextLine());
        System.out.print("Email: ");
        datosRegistro.put("Email", getS().nextLine());
        System.out.print("Contraseña: ");
        datosRegistro.put("Contraseña", getS().nextLine());
        System.out.print("Dirección: ");
        datosRegistro.put("Direccion", getS().nextLine());
        System.out.print("Descripción: ");
        datosRegistro.put("Descripcion", getS().nextLine());
        System.out.print("Distancia máxima: ");
        datosRegistro.put("DistanciaMax", getS().nextLine());
        System.out.print("Tarifa: ");
        datosRegistro.put("Tarifa", getS().nextLine());
        System.out.print("Documentación: ");
        datosRegistro.put("Documentacion", getS().nextLine());
        System.out.print("IBAN: ");
        datosRegistro.put("IBAN", getS().nextLine());
        //System.out.print("Foto: "); No es un atributo, posiblemente eliminar lineas
        //datosRegistro.put("Foto", getS().nextLine());
        return datosRegistro;
    }
}
```

```
1 usage  ▲ Mikkell Alexander Andrango Ushiña
public HashMap<String, String> loginCuidador() {
    HashMap<String, String> datosLogin = new HashMap<>();
    System.out.println("Por favor, introduce tus credenciales para iniciar sesión:");
    System.out.print("Email: ");
    datosLogin.put("Email", getS().nextLine());
    System.out.print("Contraseña: ");
    datosLogin.put("Contraseña", getS().nextLine());
    return datosLogin;
}

4 usages  ▲ Mikkell Alexander Andrango Ushiña
public void mostrarMensaje(String mensajeError) {
    System.out.println(mensajeError);
}

6 usages  ▲ Mikkell Alexander Andrango Ushiña
public void mostrarMensajeCorrecto(String mensaje) {
    System.out.println(mensaje);
}
}
```

Aquí están los controladores de Dueño y Cuidador:

Controlador Dueño:

```
package Controladores;

import ...

6 usages  ▲ Mikkell Alexander Andrango Ushiña
public class ControlDueno {
    5 usages
    private final DuenoRepositorio duenoRepositorio;
    10 usages
    private VistaDueno gui;
    2 usages  ▲ Mikkell Alexander Andrango Ushiña
    public ControlDueno() {
        this.duenoRepositorio = new DuenoRepositorio_Map();
        gui = new VistaDueno();
    }
    1 usage  ▲ Mikkell Alexander Andrango Ushiña
    public void registrarDueno() {
        HashMap<String, String> datosRegistro = gui.registrarDueno();
        if (validarDatosRegistro(datosRegistro)) {
            Dueno nuevoDueno = new Dueno(
                datosRegistro.get("Nombre"),
                datosRegistro.get("Email"),
                datosRegistro.get("Contrasenia"),
                datosRegistro.get("Direccion")
            );
            this.duenoRepositorio.create(nuevoDueno);
        } else {
            gui.mostrarMensaje( mensajeError: "No se ha podido realizar el login, intentelo de nuevo");
        }
    }

    1 usage  ▲ Mikkell Alexander Andrango Ushiña
    private boolean validarDatosRegistro(HashMap<String, String> datos) {
```

Controlador Cuidador:

```
package Controladores;

import ...

6 usages  ▲ Mikkell Alexander Andrango Ushiña
public class ControlCuidador {
    4 usages
    private final CuidadorRepositorio cuidadorRepositorio;
    2 usages
    private final DuenoRepositorio duenoRepositorio;
    14 usages
    private VistaCuidador gui;

    2 usages  ▲ Mikkell Alexander Andrango Ushiña
    public ControlCuidador() {
        this.cuidadorRepositorio = new CuidadorRepositorio_Map();
        this.duenoRepositorio = new DuenoRepositorio_Map();
        gui = new VistaCuidador();
    }
    1 usage  ▲ Mikkell Alexander Andrango Ushiña
    public void registrarCuidador() {
        HashMap<String, String> datosRegistro = gui.registrarCuidador();
        if (validarDatosRegistro(datosRegistro)) {
            Cuidador nuevoCuidador = new Cuidador(
                datosRegistro.get("Nombre"),
                datosRegistro.get("Email"),
                datosRegistro.get("Contrasenia"),
                datosRegistro.get("Direccion"),
                datosRegistro.get("Descripcion"),
                Integer.parseInt(datosRegistro.get("DistanciaMax")),
                Float.parseFloat(datosRegistro.get("Tarifa")),
                datosRegistro.get("Documentacion"),
                datosRegistro.get("IBAN"),
                datosRegistro.get("Foto") //Quitar foto de la clase Cuidador y en el constructor que si no da error
            );
        }
    }
}
```


Aquí vemos las comprobaciones de que demuestra que funciona correctamente con la funcionalidad registro/login:

```
Bienvenido a la app de Paseando a Pancho

Elige como desea interactuar con la app:
exit
Dueno
Cuidador

> Dueno
Elige la opcion que desees:
exit
Registrarme
Logearme

> Registrarme
Por favor, introduce los datos para el registro del Dueno:
Nombre: Alejandra
Email: aliba@gmail.com
Contraseña: 1234
Dirección: Atocha
Elige la opcion que desees:
exit
Registrarme
Logearme

> Logearme
Por favor, introduce tus credenciales para iniciar sesión:
Email: aliba@gmail.com
Contraseña: 1234
Elige la opcion que desees:
exit
Logout
ListarMascotas
AltaMascota
```

```
Bienvenido a la app de Paseando a Pancho

Elige como desea interactuar con la app:
exit
Dueno
Cuidador

> Cuidador
Elige la opcion que desees:
exit
Registrarme
Logearme

> Registrarme
Por favor, introduce los datos para el registro del cuidador:
Nombre: Daniel
Email: daniel@gmail.com
Contraseña: 3456
Dirección: Embajadores
Descripción: mascota
Distancia máxima: 34
Tarifa: 50
Documentación: algo
IBAN: 1234567890
Elige la opcion que desees:
exit
Registrarme
Logearme
```

```
> Logearme
Por favor, introduce tus credenciales para iniciar sesión:
Email: daniel@gmail.com
Contraseña: 3456
Elige la opcion que desees:
exit
Logout
ListarMascotas
CreacionReserva
```

Conclusión:

En este proyecto, desarrollamos “Cuidando a Pancho” utilizando Java (IntelliJ). Creemos que hemos podido cumplir los objetivos planteados y hemos verificado que su funcionamiento es correcto.

Realizamos las pruebas de caja negra y aceptación, confirmando que la implementación de las funciones registro y login son correctas.

Además, hemos asegurado que los datos cumplen con los requisitos definidos, por supuesto, se han realizado las comprobaciones.